

Gain calibration with DPPP ¹

The most common gain calibration procedures can be [performed](#) with DPPP. Formerly [BBS](#) was the standard calibration tool, but for the supported calibration scenarios DPPP is at least 10 times faster. For calibration scenarios that are not (yet) supported in DPPP, it may be necessary to resort to BBS, see the chapter on [BBS](#).

The calibration problem that DPPP can solve is the following: find a set of Jones matrices $\mathbf{G}_p$ (one for every station p) which corrects the measured visibilities $\mathbf{V}_{pq}$ to closely resemble the model visibilities $\mathbf{M}_{pq}$ (for all baselines pq), i.e. minimize

$$\|\mathbf{V}_{pq} - \mathbf{G}_p \mathbf{M}_{pq} \mathbf{G}_q^H\|$$

The matrices $\mathbf{G}$ will be referred to as **gain matrices** although they correct for more than just electrical gains.

Calibration variants

There are various options to restrict the shape of $\mathbf{G}$. The main difference is whether to solve for the amplitude of the solutions or only for the phase. Also, the number of free parameters can be restricted.

Shape	Calibration type	Free parameters
$\mathbf{G}_p = \begin{pmatrix} A_{xx}^{(p)} e^{\phi_{xx}^{(p)}} & A_{xy}^{(p)} e^{\phi_{xy}^{(p)}} \\ A_{yx}^{(p)} e^{\phi_{yx}^{(p)}} & A_{yy}^{(p)} e^{\phi_{yy}^{(p)}} \end{pmatrix}$	fulljones	8
$\mathbf{G}_p = \begin{pmatrix} A_{xx}^{(p)} e^{\phi_{xx}^{(p)}} & 0 \\ 0 & A_{yy}^{(p)} e^{\phi_{yy}^{(p)}} \end{pmatrix}$	diagonal	4
$\mathbf{G}_p = \begin{pmatrix} e^{\phi_{xx}^{(p)}} & 0 \\ 0 & e^{\phi_{yy}^{(p)}} \end{pmatrix}$	phaseonly	2
$\mathbf{G}_p = \begin{pmatrix} e^{\phi^{(p)}} & 0 \\ 0 & e^{\phi^{(p)}} \end{pmatrix}$	scalarphase	1

Making a skymodel

To perform a calibration, you need a sky model (see section [Source catalog](#) for details). You can get one from the catalogs in **gsm.py** (see section on [GSM](#) for details), or make your own. To make the sky model (in text format) readable by DPPP, it needs to be converted from plain text to a **sourcedb**. That is done with the program **makesourcedb**. Usually the sourcedb is called 'sky' and copied into the data set you're reducing. If you put it elsewhere, it is customary to give it the extension **.sourcedb**.

```
makesourcedb in=my.skymodel out=L123.MS/sky format='<'
```

The part **format='<'** is necessary to convince makesourcedb that the format is given by the first line in the file. You can use the program **showsourcedb** to verify the output sourcedb.

It is also possible to calibrate on model visibilities - in this case no sky model is necessary. See the online documentation of DPPP, the parameter to look for is **usemodelcolumn**.

Calibration

To perform a phase only calibration, the following parset can be given to DPPP.

```
msin=L123.MS
msout=
steps=[gaincal]
gaincal.sourcedb=L123.MS/sky
gaincal.parmdb=L123.MS/instrument
gaincal.caltype=phaseonly
gaincal.solint=2
gaincal.nchan=0
```

The part **solint=2** specifies that we only want one solution for every two time intervals. This can improve the signal to noise ratio - but one should have a physical argument that tells that the solutions do not change within the solution interval. The part **nchan=0** tells that one solution is computed that is assumed constant over the entire band. Setting it to e.g. **nchan=1** will compute a separate solution for each channel (again, at the cost of signal to noise).

The parset above performs a phase only calibration, and stores the calibration result in the **parmdb** (parameter database) **L123.MS/instrument**. This file will be created if it is not there yet. If it does exist, the solution in it will be overwritten. Note that it is a convention to save the calibration tables in a file (casa table) called **instrument** in the data set being reduced. If you store it outside the data set, the convention is to give it the extension **.parmdb**.

The solution table can (and should!) be inspected with `parmdbplot.py`, see Section~ref{bbs:inspect-solutions}. Note that this calibration step does not yet change the visibilities. To perform complex operations on the solutions, like smoothing them, LoSoTo can be used, see Chapter~ref{losoto}.

Applying solutions

To apply the calibration solutions in DPPP, the step **applycal** can be used. The following parset applies the solutions that were obtained by `gaincal`. Note that currently, the solution table is only written at the very end of DPPP, so that it is not possible to solve and apply the solutions in the same run of DPPP.

```
msin=L123.MS
msout=.
msout.datacolumn=CORRECTED_DATA
steps=[applycal]
applycal.parmdb=L123.MS/instrument
```

It is a convention to write the output to the column **CORRECTED_DATA**, to avoid changing the original data column **DATA**.

Transferring solutions and the beam

When transferring solutions from a calibrator to a target, the sensitivity of the beam across the sky needs to be taken into account: the instrument does not have the same sensitivity at the position of the calibrator field as at the position of the target field. You can compensate for this by using a model for the LOFAR beam. Effectively, then instead of equation~ref{eq:dpppcal} the following equation is solved for $\mathbf{G}_p$:

$$\|\mathbf{V}_{pq} - \mathbf{G}_p \mathbf{B}_p \mathbf{M}_{pq} \mathbf{B}_q^H \mathbf{G}_q^H\|$$

In the case of transferring solutions, the calibration is usually about the amplitude and the calibration type should be either **diagonal** or **fulljones**.

A parset for calibrating on the calibrator field, taking the beam into account, is given below:

```
msin=L123.MS
msout=
steps=[gaincal]
gaincal.sourcedb=L123.MS/sky
gaincal.parmdb=L123.MS/instrument
gaincal.caltype=diagonal
gaincal.usebeammodel=true
```

Applying the beam

When applying the solutions of the calibrator to the target, you should probably not apply the beam, so that another round of calibration is possible afterwards. Only after you are done with all calibration, the beam should be applied (just before imaging). Applying the beam is possible with

```
msin=L123.MS  
msin.datacolumn=CORRECTED_DATA  
msout=.  
msout.datacolumn=CORRECTED_DATA  
steps=[applybeam]
```

Footnotes

[1] : This section is maintained by [Tammo Jan Dijkema](#).